

ENGINEERING BRIEF

The memory behind the agent.

A multi-tenant knowledge hub. Staff feed it, the AI agent serves it live, and every conversation flows back as memory. Built for Flare Fitness. Live in production.

PREPARED FOR

The CTO

STATUS

Live in production

ENDPOINT

brain-
flare.vercel.app

DATE

July 2026

The Brain holds a customer's business knowledge, structured and versioned, and serves it live to the AI agent that answers their customers. For Flare Fitness that agent is **FAI**, on WhatsApp and Instagram, in Kuwaiti Arabic.

2

Halves of one loop, both live: humans feed, agents serve and file back

15

Named agent operations at one self-describing endpoint

623⁺

Live knowledge entries across 19 typed modules

94_k

Flare conversation threads the system learns from

0

Tokens spent detecting complaints and churn signals

100%

Tenant isolation enforced in Postgres, not app code

THE SHAPE OF IT

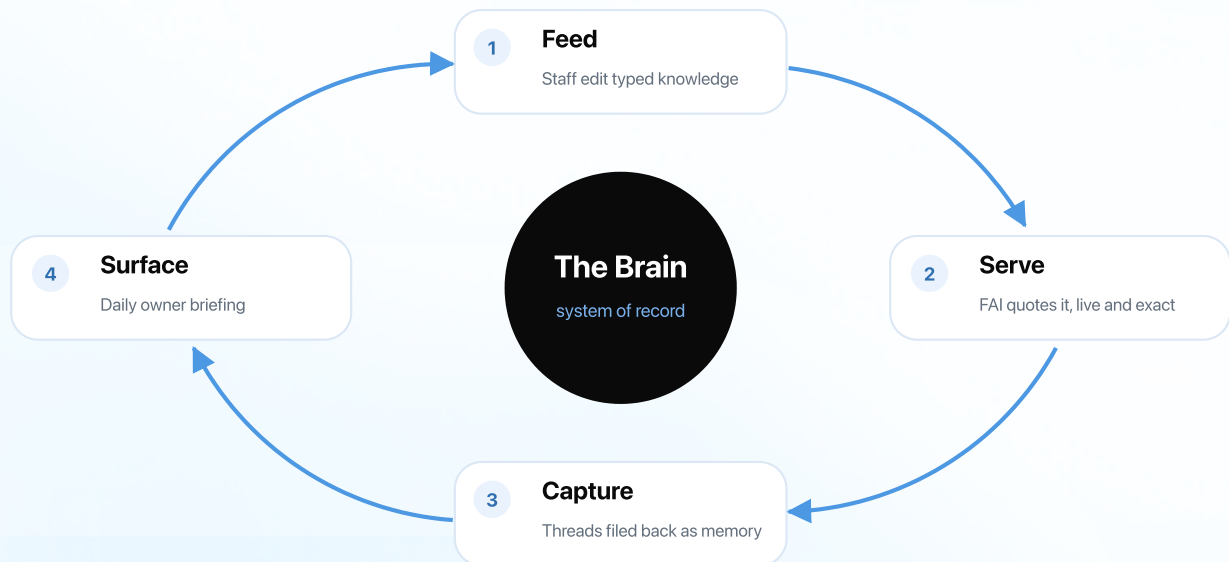
Staff feed structured knowledge. The agent serves it live, then files each conversation back as typed memory. The archive stops being dead storage and starts working.

ONE ACTION ITEM

The production admin account still carries seeded credentials on a public URL. **Rotate it.** Everything else on the list is a planned upgrade, not a defect.

An agent with no memory treats every conversation as the first. The Brain closes the loop, in four stages.

THE KNOWLEDGE LOOP



1 Feed

A React dashboard. 19 modules, 623+ typed entries, every change versioned.

2 Serve

FAI reads knowledge on nearly every message and never invents a price.

3 Capture

One call files the customer, the transcript, and the detected signals.

4 Surface

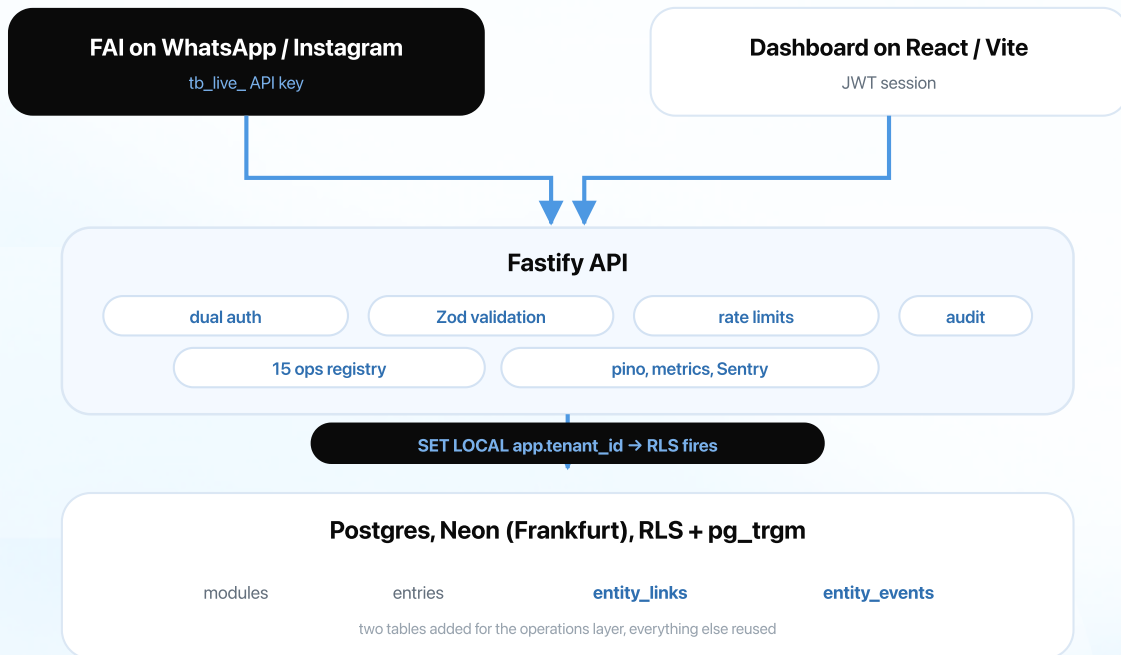
Accumulated memory becomes an early-warning digest the owner feels.

BEFORE THE BRAIN

Prices lived in people's heads, so the agent went stale the moment they changed. Every finished conversation vanished. A member could complain on Monday and be a stranger by Thursday.

A pnpm monorepo: a Fastify API in front of Postgres, a React dashboard, a prompts package. Deliberately conventional, so the interesting parts stay legible.

SYSTEM TOPOLOGY



One catchall on Vercel

Bracket files match a single path segment, so all routing re-injects into the full Fastify app through `api/v1/catchall.ts`.



Contract defined once

`packages/shared` holds the Zod schemas used by both API and web, so the data contract is written in exactly one place.



Redis is optional

The cache client stubs to no-ops when unset and fails fast when down. The system runs correctly with no Redis at all.



External keys parked

Anthropic, OpenAI, SMTP, and R2 carry placeholders. The features needing them are deferred, not broken.

One file defines every operation: name, scope, params, handler. The REST routes, permissions, and docs all follow from it. This is how the agent talks to the Brain.



What every call passes through



1 Auth

`tb_live_` keys hit a SHA-256 lookup with revocation and expiry. Otherwise JWT.



3 Validate + tenancy

Zod checks params, then the handler runs inside the tenant transaction with RLS active.



2 Scope

read, write, or admin. Agent keys never reach admin. A 403 names the missing scope.



4 Audit

Every write and admin call writes an `audit_log` row, payloads truncated to counts.

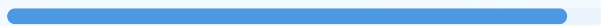
One `ingest_conversation` call files a finished thread. In one transaction: upsert the customer, store the transcript, link them, log events, detect signals.

Three signals, detected on Kuwaiti Arabic

complaint	KEYWORD	churn_risk	KEYWORD	sales_intent	KEYWORD
مشكلة استرجاع	ما اشتغل	الغاء بطل الاشتراك		سعر خصم كود	

Deliberately tuned high-precision

Precision high



A false complaint alert erodes the owner's trust in the morning briefing.

Recall low, on purpose



A missed signal is recoverable. When we want recall, an LLM pass slots in behind the same contract.

0

Tokens per detection

0_{ms}

Added latency

0

Hallucinated complaints, ever

REAL PRODUCTION RESULT

A real thread, a 33% code that applied as 10%: customer upserted, **escalated: true**, signals [**complaint, sales_intent**]. Re-ingesting the same thread returns the same ids. Idempotent.

Tenant isolation lives in the database, not in application code. Defense in depth, from the edge to the row.

-  **Edge.** Helmet CSP and HSTS, per-route rate limits, constant-time login throttle.
-  **Auth.** JWT for users, tb_live_ keys for agents, SHA-256 hashed, least-privilege.
-  **Validation + tenancy.** Zod on every handler, FORCE ROW LEVEL SECURITY on every table, no BYPASSRLS.
-  **Audit.** Every write and admin call logged with operation name and truncated params.

a buggy query cannot see another tenant's rows

ACTION REQUIRED, TODAY

The production **PAIR_ADMIN** account still carries the seeded credentials from [infra/seed/](#), on a public URL, and can mint API keys for any tenant. **Rotate it now, with the seeded editor account.**

Supply chain and hygiene

- ✓ Gitleaks secret scan on every push
- ✓ `pnpm audit` at high severity on every push
- ✓ Coverage thresholds ratchet up in CI
- ✓ Runbooks: auth, RLS leak, DB failover, LLM outage

What is live, what is next, and what comes later. Every limit has a contained upgrade path.

LIVE NOW	3	NEXT	2	LATER	2
Both halves of the loop feed + serve + capture		Vector retrieval pgvector + embeddings, Kuwaiti dialect		LLM signal pass recall, behind same contract	
15 operations + RLS contract-first, audited		Seed cleanup + timezone small, isolated fixes		MCP surface generated from the same registry	
Arabic signal detection keyword, high-precision					

THE ONE THAT MATTERS

Embeddings first. Retrieval is keyword-only today, so Arabic morphology can miss (اشترالك matches, الاشتراك can slip). It is the one limit a customer feels, and the fix sits behind the [query](#) operation the agent already calls.

Verified on production, with real data

- ✓ health: 19 modules, 623 active entries
- ✓ ingest on a real Kuwaiti-Arabic thread
- ✓ graph_query customer to conversation
- ✓ daily_briefing full digest
- ✓ scope enforcement returns 403 named
- ✓ idempotent re-ingest, same ids

The system runs on our own stack, git-connected and auto-deploying. Picking it up is a short list.

1

Push to [main](#) to ship, no CLI ritual

2

Steps to wire an agent: load resolver, hand it a key

0

Docker needed for local dev, embedded Postgres boots itself

To pick it up

- 1 Clone [trypairai-ally/the-brain](#), run `pnpm dev` for the full loop locally against embedded Postgres on port 55433.
- 2 To ship, push to [main](#). Vercel auto-builds and deploys to [brain-flare.vercel.app](#). Neon Postgres sits in Frankfurt.
- 3 To wire FAI, load [skills/RESOLVER.md](#) into its system prompt and hand it a scoped `tb_live_` key. The skills already point at production.
- 4 First, rotate the seeded admin credentials. Then embeddings behind the [query](#) operation.

TWO-LAYER DESIGN

A machine-checkable operations contract underneath, human-readable skills on top. The agent's behavior is reviewed in a pull request, not buried in code.

TWO LATENT BUGS FOUND AND FIXED

Arabic POST bodies were 400-ing through the Vercel catchall (stale `content-length` on multibyte text). A Redis outage could starve the 5-second transaction. Both fixed on [main](#).



The archive stops being dead storage. It gets smarter every day it runs.

Live in production. One thing to do on the way in: rotate the seeded admin credentials. One thing next: embeddings.

Built in Kuwait. Paired, not queued.